

PARTE I – Criando índices e analisando os planos de execução usados pelo otimizador de consulta

Para essa prática, vamos usar a ferramenta Oracle SQL Developer

1. Crie a tabela CLIENTES.

```
CREATE TABLE CLIENTES
( id number(10) GENERATED BY DEFAULT AS IDENTITY primary key,
  nome varchar(40),
  uf varchar(2)
);
```

2. Carregue 1 milhão de linhas na tabela.

```
INSERT INTO clientes (nome, uf)
SELECT dbms_random.string('U',trunc(dbms_random.value(1,10))),
       case when mod(level,100000) = 0 then 'AC'
            when mod(level,10000) = 0 then 'PB'
            else 'SP'
       end
FROM dual
CONNECT BY level <= 1000000;

COMMIT;
```

3. Selecione todos os clientes que começam com FA

```
SELECT * FROM clientes WHERE nome LIKE 'FA%';
```

4. No menu, use o botão “Plano de Explicação” (Explain plan) para visualizar como o otimizador de consulta executou o comando.

➔ Observe a opção usada pelo otimizador para acessar os dados assim como a CARDINALIDADE e O CUSTO

De maneira breve e resumida:

- Cardinalidade: é o número estimado de linhas que a instrução retornará.
- Custo é a quantidade estimada de tarefas executadas pelo plano para que o dado seja retornado.

Worksheet: Explain Plan... (F10)

SQL | 0.068 seconds

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT				726
TABLE ACCESS	CLIENTES	FULL	1424	726
Filter Predicates				
NOME LIKE 'FA%'				
Other XML				

5. Crie um índice chamado CLIENTES_NOME_IDX na coluna NOME da tabela CLIENTES e verifique o comportamento do otimizador

```
CREATE INDEX clientes_nome_idx ON clientes(nome)
ONLINE COMPUTE STATISTICS;
```

Verique o plano de execução da consulta e note que o otimizador pode não ter todas informações (estatísticas) dos dados para dar o melhor plano.

Worksheet: Query Builder

SQL | 0.017 seconds

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT				726
TABLE ACCESS	CLIENTES	FULL	1175	726
Filter Predicates				
NOME LIKE 'FA %'				

Atualize as estatísticas da tabela

```
EXEC DBMS_STATS.gather_table_stats(user, 'CLIENTES', cascade=>true);
```

Verifique novamente plano de Execução:

Worksheet: Query Builder

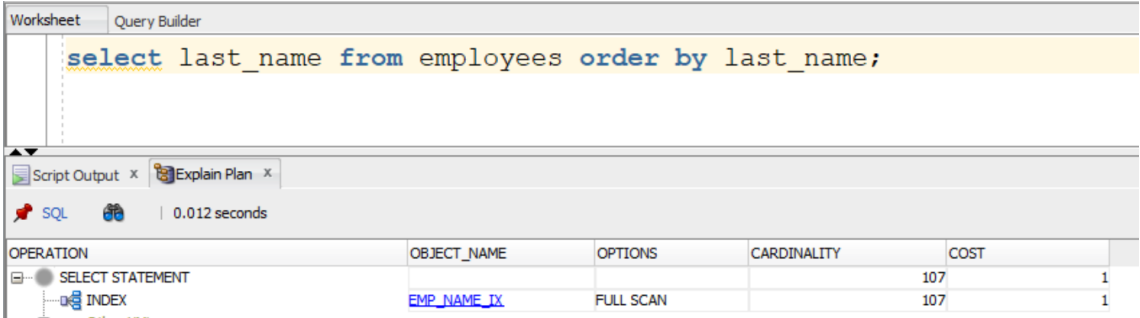
SQL | 0.014 seconds

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT				54
TABLE ACCESS	CLIENTES	BY INDEX ROWID BA...	54	54
INDEX	CLIENTES_NOME_IDX	RANGE SCAN	54	3
Access Predicates				
NOME LIKE 'FA %'				
Filter Predicates				
NOME LIKE 'FA %'				

➔ Note como o uso do índice diminui drasticamente o custo da operação.

6. Agora verifique o plano de execução da consulta abaixo que não utiliza cláusula where para filtrar dados

```
select last_name from employees order by last_name;
```



OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			107	1
INDEX	EMP_NAME_IDX	FULL SCAN	107	1

➔ Note que como índice é armazenado de maneira ordenada e a consulta precisa mostrar apenas o last_name – que está no índice – o otimizador não precisa ler a tabela employees e consegue executar o comando de maneira muito mais rápida lendo apenas o índice, que é bem menor do que a tabela 😊

PARTE II – Otimizando criação de índice em grandes tabelas (grande volume de dados)

1) Remova o Índice criado na prática anterior

```
DROP INDEX CLIENTES_NOME_IDX;
```

2) Habilite a opção de mostrar o tempo de execução de um comando no SQL Developer execute o comando para criar o índice novamente

```
SET TIMING ON
```

```
CREATE INDEX clientes_nome_idx ON clientes(nome) ;
```

3) Remova o Índice novamente

```
DROP INDEX CLIENTES_NOME_IDX;
```

4) Agora recrie o mesmo índice usando opção de paralelismos:

```
CREATE INDEX clientes_nome_idx ON clientes(nome) PARALLEL 5;
```

➔ Note que o tempo de criação do índice diminui drasticamente. Para tabelas com grande volumes de dados, paralelismo deve sempre ser levado em consideração na hora de criar índices

PARTE III – Índice B-tree x Bitmap

1) Crie o índice B-tree na coluna UF da tabela clientes

```
CREATE INDEX clientes_uf_idx ON clientes(uf) online;
```

2) Atualize as estatísticas da tabela clientes e de seus índices

```
EXEC DBMS_STATS.gather_table_stats(USER, 'CLIENTES', cascade=>true);
```

3) Verifique o plano de execução de cada consulta abaixo. Atende para cardinalidade e custo

-- clientes do Acre ou da Paraíba

```
SELECT * FROM clientes WHERE UF IN ('AC','PB');
```

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			100	5
INLIST ITERATOR				
TABLE ACCESS	CLIENTES	BY INDEX ROWID BA...	100	5
INDEX	CLIENTES_UF_IDX	RANGE SCAN	100	4

-- Agora clientes de São Paulo

```
SELECT * FROM clientes WHERE UF = 'SP';
```

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			999900	725
TABLE ACCESS	CLIENTES	FULL	999900	725

Note que como a maior parte das linhas da tabela tem o valor da coluna UF = 'SP', mesmo existindo um índice na coluna UF ele opta por fazer um acesso FULL na tabela, pois realmente é mais rápido neste caso.

4) Verifique o tamanho do índice

```
select segment_name,segment_type, blocks, bytes/1024 KBytes  
from user_segments  
where segment_name = 'CLIENTES_UF_IDX';
```

	SEGMENT_NAME	SEGMENT_TYPE	BLOCKS	KBYTES
1	CLIENTES UF IDX INDEX		2048	16384

5) Remova o Índice CLIENTES_UF_IDX

```
DROP INDEX CLIENTES_UF_IDX;
```

6) Recrie o índice mas agora como um BITMAP index

```
CREATE BITMAP INDEX clientes_uf_idx ON clientes(uf) ;
```

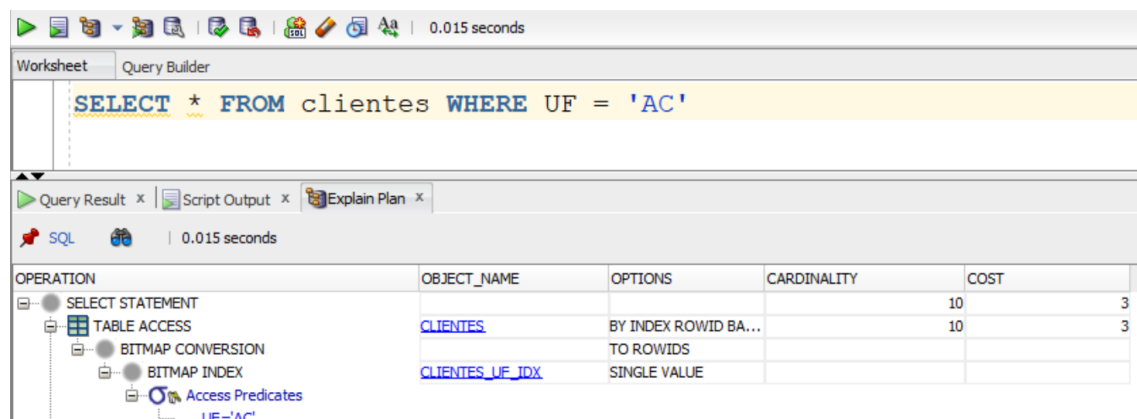
7) Atualize as estatísticas da tabela clientes e de seus índices

```
EXEC DBMS_STATS.gather_table_stats(USER, 'CLIENTES', cascade=>true);
```

8) Verifique o plano de execução de cada consulta abaixo. Atende para cardinalidade e custo

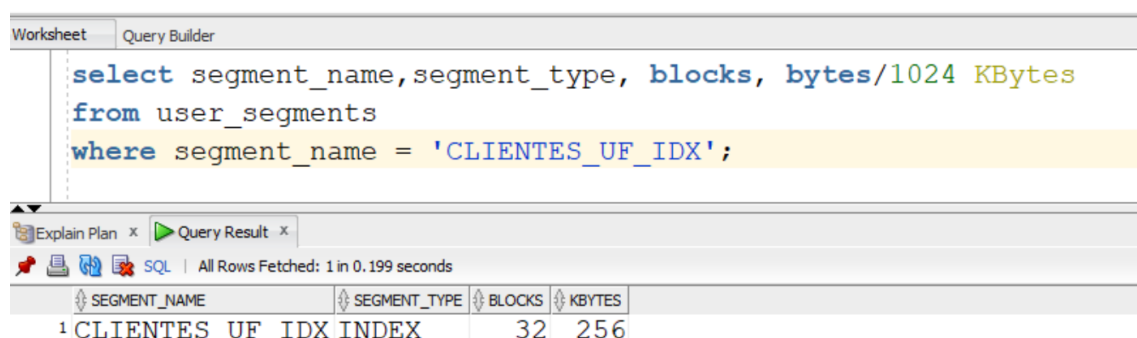
-- clientes do Acre ou da Paraíba

```
SELECT * FROM clientes WHERE UF = 'AC'
```



OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT				10 3
TABLE ACCESS	CLIENTES	BY INDEX ROWID BA...	10	3
BITMAP CONVERSION		TO ROWIDS		
BITMAP INDEX	CLIENTES_UF_IDX	SINGLE VALUE		
Access Predicates		UF='AC'		

9) Verifique o tamanho do Índice



SEGMENT_NAME	SEGMENT_TYPE	BLOCKS	KBYTES
1 CLIENTES UF IDX INDEX		32	256

Compare com o tamanho do índice quando ele foi criado com o tipo B-Tree

- ⇒ Lembre-se que índice BITMAP gera alto nível de bloqueio na tabela, o que não é desejado em sistemas transacionais. Devem ser tipicamente usados em DW or sistemas onde apenas uma transação faz carga de dados na tabela e todas outras apenas consultam